Developing an AI model for Q&A on Financial report of an organisation

Durgesh Bhirud

Sr. No.	Table of Contents	Page No.
1	Introduction, Problem History and Technological Challenges	3
1.1	Introduction	3
1.2	Problem History	3
1.3	Technological Challenges	3
2	Data Analysis	4
3	Recommended Models and Technology Stack	5
4	Model Architecture, Solution Pipeline and the Interface	6 - 9
4.1	Model Architecture	6 - 7
4.2	Solution Pipeline	8
4.3	Gradio Interface	9
5	Model Fine Tuning, Optimization and Relevant Analysis	10 - 13
5.1	Model Fine Tuning, Optimization	10 - 11
5.2	Result Analysis	12-13
6	Example test run of the model	14
7	Business Impact	15
8	Future Enhancements	16
9	References	16

1. Introduction, Problem History and Technological Challenges

1.1 Introduction:

This project is focused on developing a question-answer (QA) system for financial reports using a combination of AI models and document processing tools. The system aims to extract meaningful insights from financial reports, and answer user queries effectively. These reports can be submitted by the user or the user can choose from any of the already uploaded reports of some of the top companies of the world.

1.2 Problem History:

The extraction and interpretation of financial data from documents such as PDFs, which often contain complex data like tables, narratives, etc. have long been a challenge in the finance industry. Traditionally, financial analysts have manually gone through these reports to gather data, which is time-consuming, labour-intensive, and prone to errors. Moreover, answering specific queries from large volumes of reports requires in-depth knowledge and significant human effort, so anyone with a non-financial background may find it difficult to understand the reports and will need some external help for their tasks.

1.3 Technological Challenges faced in solving this problem:

Several technical hurdles are involved in building a QA system for financial reports:

- **Document Structure Complexity:** Financial reports are heterogeneous, containing text, tables, etc. parsing and interpreting such mixed data types is challenging.

- **Table Extraction:** Extracting structured data from tables requires specialised table extraction techniques.

- **Handling follow up questions:** The chatbot should be able to handle follow up questions, which means it should also consider previous query-response pairs to formulate the answers.

- **Scalability:** The system must be able to handle numerous users querying large volumes of reports in real-time without significant latency.

2. Data Analysis

The report consists of different types of data say from texts to tables. The textual data in the financial reports consists of narratives, descriptions, financial terms, etc. Structured data like the tabular data within the report contains financial metrics like revenue, profit margins, etc.

Due to different types of data available in the report, it is important to use the appropriate tools and libraries for data extraction.

Text Extraction: The reports are submitted in a pdf format, so the text is extracted using the **PyPDF library** which is further broken down into chunks to be embedded for context retrieval to answer user queries.

Table Extraction: The tables present in the reports also contain useful information, in order to extract this data, libraries like **Camelot** are used. Similar to the text chunks this is then embedded for context retrieval to answer user queries.

It is important to have **Data Integrity** which is to ensure that the data extracted from tables and texts is accurate and in general there is no loss of numerical data or any kind of data. This is crucial for the QnA system because accurate, reliable data ensures that extracted insights and responses are correct. Errors or inconsistencies in the underlying data (e.g., PDFs, tables) could lead to faulty conclusions, misinterpreted financial metrics, and compromised decision-making, affecting user trust and business outcomes.

3. Recommended Models and Technology Stack

There are a plethora of models and libraries available to us for building such a QnA bot. The recommended models here are:

Language models: GPT 3.5 and GPT 4, these models are predominantly used for chat based tasks, as they are effective in natural language understanding and generation. GPT-3.5 offers strong performance across diverse tasks, while GPT-4 improves upon accuracy, context handling, and reasoning capabilities for more complex queries and tasks. For their diverse functionalities, these models will be used for the answer generation as well as for the answer evaluation and explanation.

Embedding models: Azure OpenAI Embeddings enable powerful search, semantic understanding, and natural language processing by converting text into numerical vectors, making it easier to perform similarity searches or clustering tasks. It leverages OpenAI's models, integrated with Azure's cloud services, to provide scalable and efficient embedding generation.

Different Technological Stacks required for this project are:

Reading the PDF reports: In order to understand the reports and extract the data from them, libraries like PyPDF and Camelot are necessary as they take care of the textual and the tabular data respectively.

Embeddings: Now embedding this data and storing them in a vector database can be done with the help of ChromaDB, from where the necessary documents can be extracted to generate a response.

OpenAI: Language models like GPT 3.5 and GPT 4 are some of the best available language models for the task of response generation and evaluation.

Gradio: In order to have an interface to interact with the model i.e. the place where the uploading of PDF, also where question and answers are visible, it is important to use Gradio or Streamlit.

Azure: Azure gives access to different tools for our use and also or additional text analytics services (like sentiment analysis, OCR, etc.).

4. Model Architecture, Solution Pipeline and the Interface

4.1 Model Architecture

The basic architecture of the model is as follows:

1) **PDF/ Report Processing :** First step is to handle the submitted pdf and loading the data from it in our system. For this task we have two tools:

   PyPDF - Extracts text from the PDF pages by reading the content using the loader.

   Camelot - Extracts the tabular data from the PDF pages, converting them into a text format or dataframe structure.

   So this uploaded PDF is parsed into text chunks and table chunks, and also we can control the chunk size and chunk overlap while loading the pdf. Text chunks are split using a text-splitting algorithm to ensure they are manageable, while table data is extracted into a structured format for further processing.

2) **Embeddings layer :** This layer takes the extracted text and table data from the report and converts them into vector representations also known as embeddings.

   This task is done with the help of the AzureOpenAIEmbeddings, which is a service provided by the Azure OpenAI, it generates vector embeddings of text inputs, which are numerical representations that capture the semantic meaning of the text.

   Now these embeddings are taken and stored in a ChromaDB vector database, which allows for a faster retrieval of documents when queried.

   In addition to these embeddings, there are embeddings of financial reports of some big organisations already available for use in persisted vector databases. This is done so that an user without a report can still perform the QnA task on the already available reports of certain organisations.

3) **Querying layer :** Based on the user query and the past conversation history, the necessary documents are queried/retrieved from the vector database to provide the necessary context for response generation.

   This retrieval is done with the help of similarity search in ChromaDB which uses cosine similarity to get the appropriate documents or we can say the embeddings from the vector database. Cosine similarity looks at the angle between the two vectors as a criteria for their similarity. Lesser the angle between the two vectors, higher is the similarity between them. By default the retriever extracts the five most similar documents from the database.

4) **Response Generation layer :** Now the next task of the system is to generate a valid response based on the retrieved documents. The GPT 3.5 or the GPT 4 is used for the answer generation task.

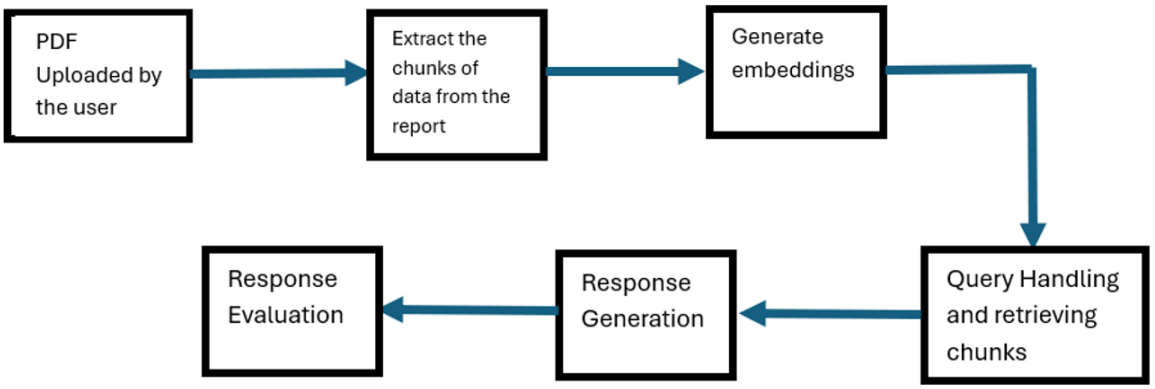
   The retrieved documents from the querying layer are taken to the GPT model as context, along with the user's question. The model then generates a natural language response based on this context, aiming to provide a relevant, coherent answer.

5) **Evaluation layer :** It is necessary to evaluate the responses generated on the basis of its accuracy, relevance, and completeness. We use a second LLM model as a scoring system to evaluate how well the generated answer aligns with the user's query and the document context. This evaluation helps in fine-tuning the system's performance.

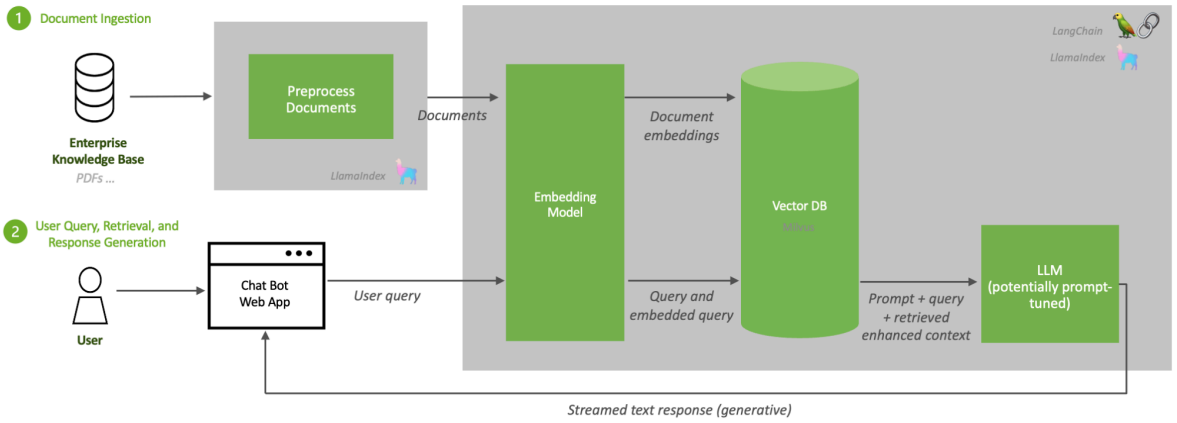
   In this case, whenever the user chooses the GPT 3.5 model for the response generation, the GPT 4 model will be used for the evaluation purpose and vice versa.

   The evaluation model will receive the question, context and the response generated by the LLM model in order to give an evaluation score to see if the response was comprehensive and relevant enough to be answer the question based on the given context.

4.2 Solution pipeline

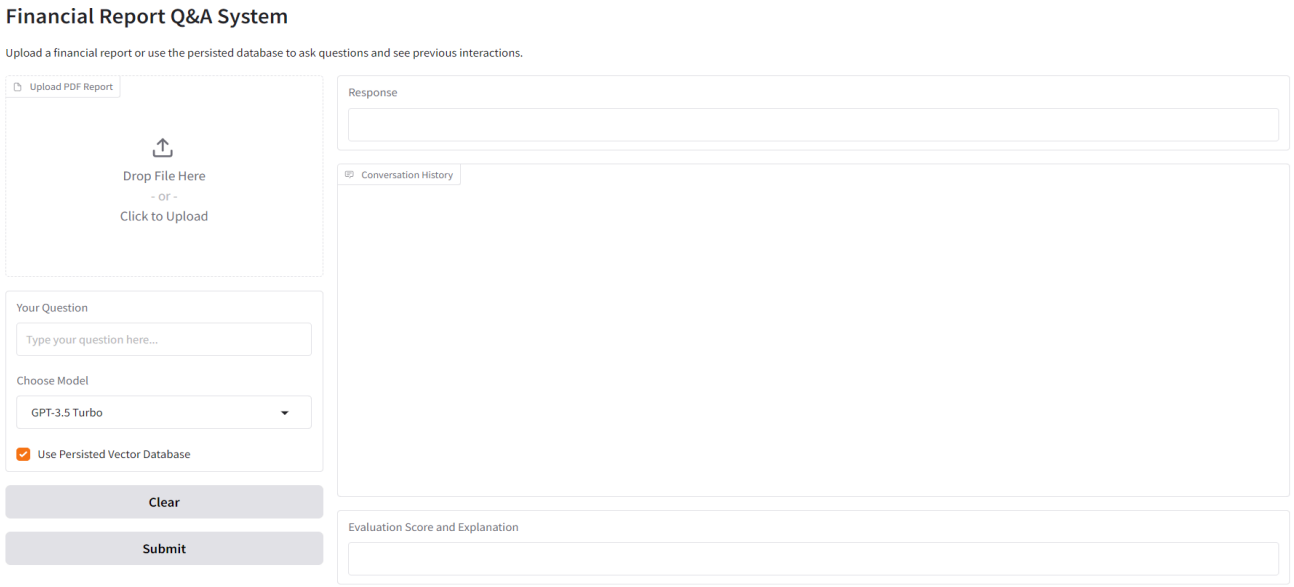


Retrieval Augmented Generation (RAG) Sequence Diagram



4.3 Interface

After getting the necessary answer, it is visible to the user through the Gradio interface Different aspects of the interface are explained here:



In the **Upload PDF report** section, the user should upload the financial report for which the QnA system will be set up. Also, you can see the **Use Persisted Vector Database** button, where the user can select the already uploaded PDF reports.
Note: In this case only one report (of Alphabet. Inc - Google) is stored as embeddings in the vector database for the user's use.

The user has the customizability to choose the model which will perform the response generation in the **Choose Model** section. In this case, the user can choose between the GPT 3.5 model and the GPT 4 model.

The user needs to type their question in **Your Question** box and then get the generated response in the **Response** section after clicking on the **Submit** button.

The **Conversation History** will track the past question-response pairs for that run.

Clear button clears all the sections, and thus the system is ready for the next run.

Evaluation Score and Explanation, this section is only for the maker of the system where they can see the evaluation score of the generated response as given by some second LLM model and explanation for giving such a score. When the user selects the GPT 3.5 model for response generation, the evaluation is done by the GPT 4 model and vice versa.

5. Model Fine Tuning, Optimization and Relevant Analysis

5.1 Model Fine Tuning and Optimization

The GPT models used for response generation here are GPT 3.5 and GPT 4, they are improved by providing an appropriate prompt (Prompt Engineering) and the hyperparameter tuning while setting up the model.

Prompt Engineering

It involves designing and refining the input prompts given to the GPT models to elicit the most accurate, relevant, and meaningful responses.

In this case, the prompts are written in such a way that the system understands the complete task assigned to it and what way or what kind of response is expected by the user.

The prompt provided to the system is :

```
qna_system_message = """
You are a specialized assistant for a financial services firm, designed to answer user queries based on annual reports.
Your goal is to analyze the provided context from financial documents and answer user questions by following a step-by-step process.
###Guidelines for answering questions:
1. **Understand the question**: Carefully read and understand the question being asked.
2. **Analyze the context**: Break down the provided context into relevant parts that answer the question.
3. **Step-by-step reasoning**: Use a chain of thought reasoning process to guide your answer. Walk through the steps needed to derive the answer.
4. **Formulate the answer**: Ensure the answer is accurate, concise, and based solely on the provided context.
5. **State if unknown**: If the answer cannot be found in the provided context, simply respond with "I don't know."
This context will begin with the token: ###Context.
The context contains references to specific portions of a document relevant to the user query.

Previous interactions will begin with the token: ###Conversation.
The conversation contains prior user questions and assistant responses that may help in answering the current query.

User questions will begin with the token: ###Question.

Your goal is to provide accurate, concise, and relevant answers.
Avoid mentioning unnecesaary information which is not related to the question while giving the response.

If the answer is not found in the context, respond "I don't know".
Always strive to maintain professionalism and clarity in your responses.
"""
```

The prompt has to be upto the point and very particular in terms of what is expected from the model. It should be structured properly. It follows a Chain of thought (CoT) reasoning to form an answer. CoT commands the model to follow a step by step approach to understand the problem and then give the most appropriate answer accordingly.

Here in the prompt, it is mentioned that the model has to follow the five guidelines while generating a response.

- Understanding the question of the user.
- Analyzing the context provided to it based on the user query.
- To follow a step by step approach as an approach to reach the answer.
- Formulate the answer.
- Dealing with unanswerable queries.

The prompt for user message :

```
qna_user_message_template = """
###Context
The following documents contain information pertinent to the question posed:
{context}

###Question
Please answer the question based on the provided context:
{question}

###Chain of Thought
1. **Understanding the Question**: Clearly identify what the user is asking.
2. **Identifying Key Information in Context**: Highlight the most relevant parts of the provided context that directly answer the question.
3. **Formulate the Answer**: Based on the key information identified, provide the correct answer.
4. **Conclusion**: Summarize your final answer concisely.

###Answer
"""
```

Here, also the aspect of **Dynamic Prompting** is added, which is adjusting prompts based on user queries or context. The past question and answer pairs are all stored in the conversation history and provided along with the necessary documents as a context so that it helps in asking questions based on the previous responses provided to the user. All in all, prompt engineering involves trying out different prompts in order to optimize the output.

Hyperparameter Tuning

Hyperparameter tuning is the process of optimizing the settings (hyperparameters) that control how a model is trained and behaves during prediction. These parameters are not learned from the data but are critical for shaping the model's performance. Looking at how the responses are affected by changing the hyperparameters, we can find out the optimal values for them which work the best for the given task.

In building this model, hyperparameter tuning is done at several stages:

Chunk-Embedding-Retrieval : First while creating the chunks, the hyperparameters of the chunk size and chunk overlap are set at 512 and 20 respectively. Then the retriever of the persisted vector database has the search type set at 'similarity' which is the cosine similarity and the number of documents retrieved 'k' is set as 5 (Default value).

Response Generation : The hyperparameters while setting up the model are kept at the following values temperature at 0.1, this ensures the response will be more factual based and deterministic, max_tokens at 500, to ensure that the model give a complete answer and nothing is left out, top_p is set at 0.9, this allows for slight variations that increase naturalness, but still keeps the answer relevant, and the penalty terms are kept at 0 to avoid redundant phrases in GPT's responses, making them clear and concise.

Response Evaluation : For the evaluation purpose the hyperparameters are set as follows temperature at 0, to ensure that during evaluation, we get consistent judgments that don't vary due to randomness, top_p is set at 1, allowing the model to consider all possible tokens for generating its response, frequency penalty is set to 0.5, as during evaluation repetition can reduce clarity and add unnecessary noise.

5.2 Result Analysis - Response Evaluation

We can compare the two models, GPT 3.5 and GPT 4, by building an evaluation model which will score the responses generated as per the user query. This ensures that the answers generated by the system are accurate, relevant, and comprehensive.

It is important to provide a well thought prompt to the evaluation model, so that the model knows that the answer is generated by an other AI model based on so and so context in response to the user query.

```
evaluation_prompt = [
    {'role': 'system', 'content': """
You are an evaluator of the quality of answers provided by an AI assistant.
Your job is to determine how well the AI response accurately answers the question, and whether it adheres to the context.
Your evaluation should consider the accuracy, relevance, clarity, and completeness of the response.

You will receive a user question, an AI response, and the context used by the AI to generate the response.

Please provide a score between 0 and 100, where:
- 100 indicates a perfect answer, relevant, and comprehensive.
- 0 indicates a completely irrelevant or nonsensical response.

Additionally, provide a brief explanation of your score, justifying your evaluation.
"""},
{'role': 'user', 'content': f"""
### User Question:
{user_input}

### AI Response:
{response_text}

### Context:
{context_for_query}

Please provide your evaluation score and explanation based on these criteria: relevance, clarity, and chain of thought reasoning.
"""}
]
```

In this way, all the necessary information is provided to the model and then it is asked to score the response on a scale of 0 to 100 where 100 means the given response is a perfect answer which is relevant and comprehensive and 0 means the given response is completely irrelevant or nonsensical based on the user query and context provided to it.

The evaluation is based on the following criterias:

- **Relevance :** This is for evaluating whether the answer addresses the user's question based on the content provided in the report.
- **Clarity :** This grades how clear is the response i.e. how is it to understand the response given by the model.A clear response should be straightforward, without jargon or ambiguity, and structured logically.
- **Chain of Thought Reasoning :** This metric evaluates the logical progression in the AI's response, ensuring that the reasoning behind the answer is well explained. It considers whether the AI has provided intermediate steps or supporting explanations that led to the final conclusion, demonstrating the reasoning process transparently.

This evaluation of the responses given by the model can help in analysing what combination of the prompts and models gives the best response to the user query.

Overall benefits of using this evaluation metrics are:

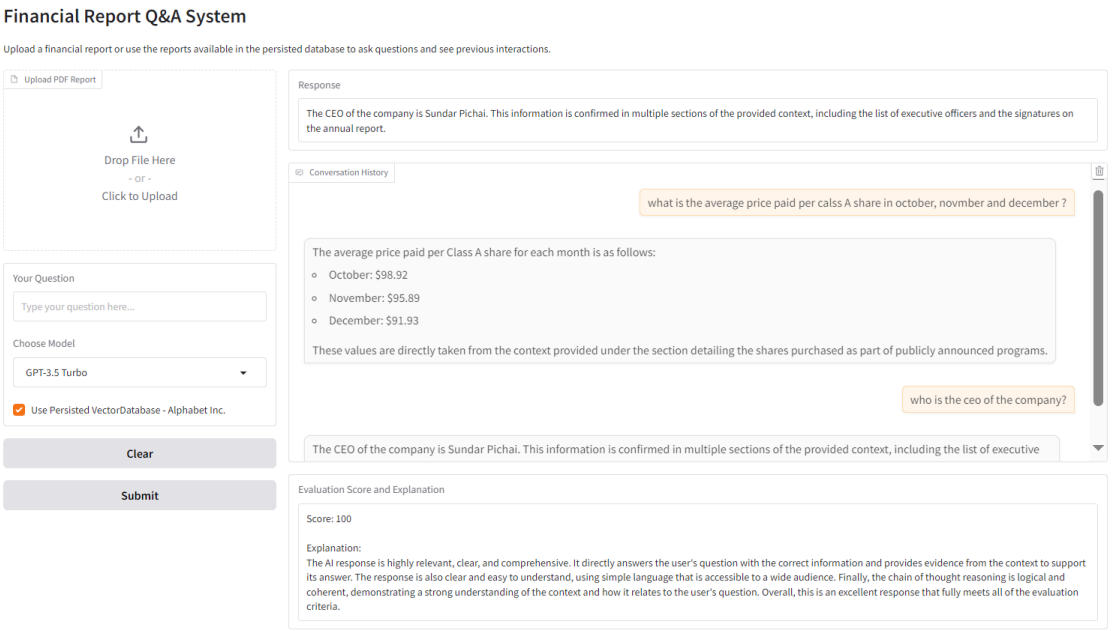
- **Diagnostic Tool:** These evaluation metrics allow you to systematically analyze the strengths and weaknesses of your model's performance. For example, if the model frequently scores low on chain of thought reasoning, you know to focus on refining the model's ability to explain intermediate steps.

- **Benchmarking and Improvement**: As you gather more evaluation data across multiple queries, you can benchmark how well your system performs over time.

- **User Feedback Loop**: Evaluation metrics provide an insight into user satisfaction. You can correlate these metrics with real user feedback to better understand if users find the system useful and whether they trust its financial insights.

- **Guiding Model Optimization**: Based on evaluation scores, you can make decisions about hyperparameters, retrieval strategies, or model types (e.g., GPT-3.5 vs. GPT-4) to optimize for your system's target performance.

6. Example test run of the model



The above image shows one of the tests run by a user.
Here we can see that the user is using the report available in the Persisted Vector Database, which is the financial report of Alphabet Inc. Google for the year 2022.

The user has chosen the GPT - 3.5 model for the response generation which will automatically set the GPT - 4 model for response evaluation and explanation.

The first question asked by the user and the response given by the model is visible in the Conversation History. For the last question asked by the user, we can see the evaluation score and the explanation for it in the bottom section.

After this the user can continue asking questions from the report or choose to clear the interface for a new run.

7. Business Impact:

If successfully implemented, this will have several positive impacts on businesses:

- **Increased Efficiency:** Automating the extraction and querying process saves time for the user and helps in quicker decision-making thus improving on the method of manually going through the report.

- **Improved Accuracy:** Building an AI model will minimise human error in reading and interpreting complex financial documents, leading to more reliable results.

- **Cost Reduction:** Reduced manual effort can lower costs for firms that deal with large volumes of financial documents.

- **Faster Insights:** Users can query financial reports and receive instant, accurate answers, which speeds up reporting and analysis workflows.

- **Scalability:** This can be scaled to handle multiple financial reports across organizations, benefiting firms that rely on large-scale financial document analysis.

8. Future Enhancements

This model can further be improved by incorporating the following things:

1) **Enhanced Chart and Graph Analysis :** Currently the model is able to extract insights from the report's textual and tabular data, but fails in properly extracting the information present in the charts or graphs which may be present in the report.

2) **Generating visual responses :** Right now the model provides textual responses to the user queries, but in financial reports it is possible for the user to have some visual representation of certain trends present in the document. So if possible this can be a useful feature addition to the model.

3) **Better memory handling :** The current model fails at properly extracting the relevant documents when the size of conversation history gets bigger. This can be improved by controlling the number of tokens being passed for response generation and by summarizing the previous conversation history and then providing this as the context instead of passing the whole conversation history.

9. References

- *Building a Q&A Bot over Private Documents with OpenAI and LangChain | by Sriram Parthasarathy | Towards AI*
- *An LLM Benchmark for Financial Document Question Answering - by : Samir Dutta and Andrei Dumitrescu*
- *RAG Based Question Answer System by Jahnavisrilakshmi Yannam*